

COMS 4732 — COMPUTER VISION 2

HW5A — The Power & of Diffusion Models and HW5B — Flow Matching from Scratch

Prof. Aleksander Holynski

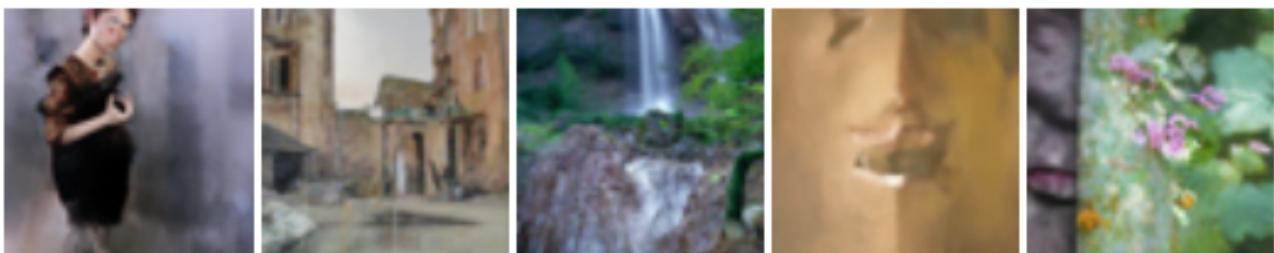
Vibhav Kashyap - vk2581

PART A • SECTION 1.5

Diffusion Model Sampling

Starting from random noise and running the iterative denoising loop produces samples directly from the pretrained diffusion model. The figure below shows the sampled outputs requested for Section 1.5.

Sampled Images



Samples generated from pure noise using the diffusion sampling loop.

`output/partA/1.5_image_stack.png`

PART A • SECTION 1.6

Sampling with Classifier-Free Guidance

Classifier-free guidance improves image quality by combining conditional and unconditional predictions during denoising. Compared with unguided sampling, the outputs are typically sharper and more aligned with the prompt.

Sampled Images with CFG

sample_1 sample_2 sample_3 sample_4 sample_5



Images sampled using classifier-free guidance. `output/partA/1.6_image_stack.png`

PART A • SECTION 1.7

Image-to-Image Translation

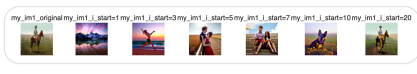
For image-to-image translation, a real image is first noised to an intermediate diffusion timestep and then denoised back. Lower noise levels preserve more of the input structure, while higher noise levels allow the model to make stronger edits.

Campanile and Additional Test Images



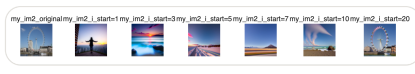
Campanile edited with the image-to-image procedure.

output/
partA/1.7_test_img.png



Additional test image 1 translated with the same procedure.

output/
partA/1.7_my_img_1.png



Additional test image 2 translated with the same procedure.

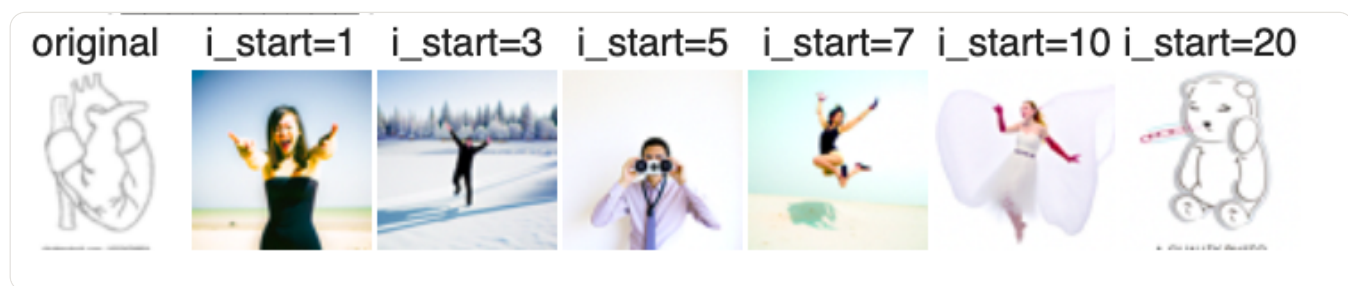
output/
partA/1.7_my_img_2.png

PART A · SECTION 1.7.1

Editing Hand-Drawn and Web Images

This section applies the same image-to-image pipeline to non-photorealistic inputs. The goal is to project sketches and a web image onto the natural image manifold while varying the noise level over values such as 1, 3, 5, 7, 10, and 20.

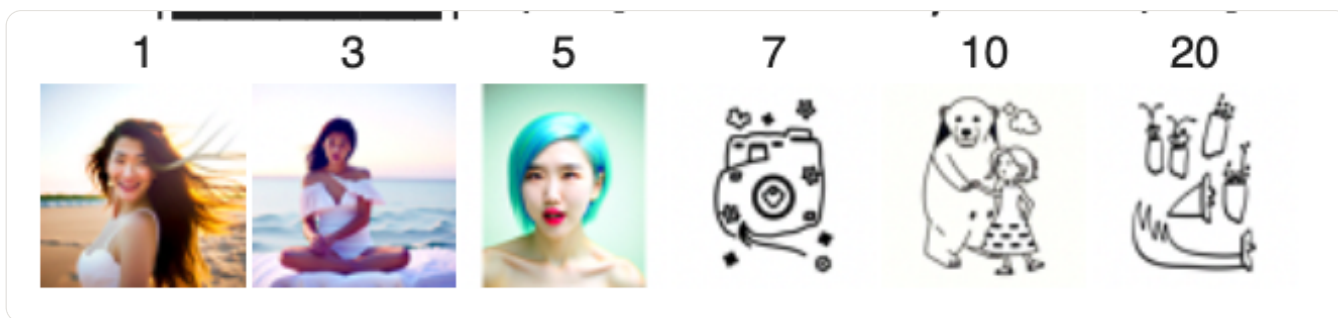
Web Image



A web image edited across multiple noise levels using the same image-to-image method.

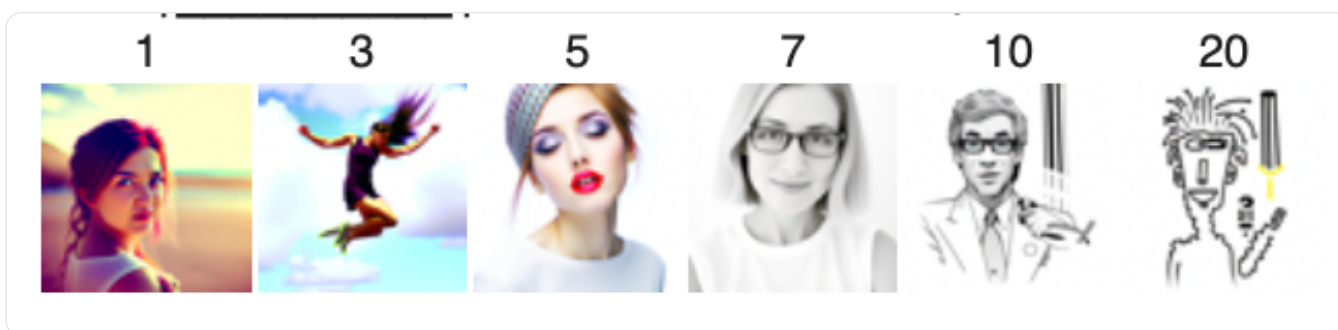
output/partA/1.7.1_web_img.png

Hand-Drawn Image 1



Hand-drawn image 1 edited over multiple noise levels. `output/partA/1.7.1_my_img_1.png`

Hand-Drawn Image 2



Hand-drawn image 2 edited over multiple noise levels. `output/partA/1.7.1_my_img_2.png`

PART A • SECTION 1.7.2

Inpainting

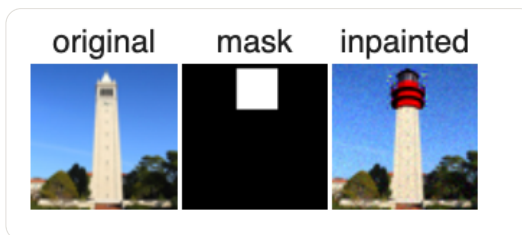
Inpainting keeps the unmasked part of the image fixed while letting the diffusion model regenerate masked regions. Below are the Campanile result and two additional masked edits.

Inpainting Results



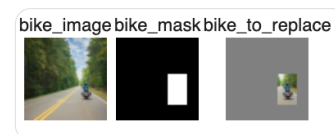
Original Campanile input.

output/
partA/1.7.2_campanile.png



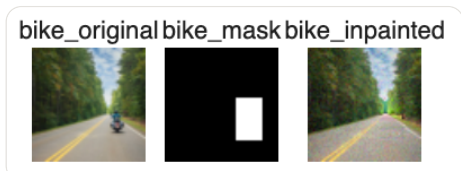
Inpainted Campanile result.

output/
partA/1.7.2_campanile_inpainted.png



Original custom image 1.

output/
partA/1.7.2_road.png



Inpainted custom image 1.

output/
partA/1.7.2_road_inpainted.png



Original custom image 2.

output/
partA/1.7.2_venice.png



Inpainted custom image 2.

output/
partA/1.7.2_venice_inpainted.png

PART A • SECTION 1.7.3

Text-Conditioned Image-to-Image Translation

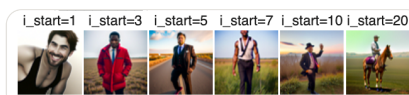
Here the denoising process is guided not only by the starting image but also by a prompt embedding. This lets the model preserve broad structure from the input while steering the output toward the requested text condition.

Text-Conditional Edits



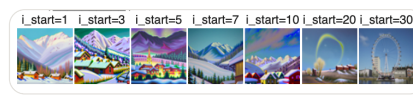
Campanile edits across multiple noise levels with text conditioning.

output/
partA/1.7.3_campanile.png



Text-conditioned edits for custom image 1.

output/
partA/1.7.3_my_img_1.png



Text-conditioned edits for custom image 2.

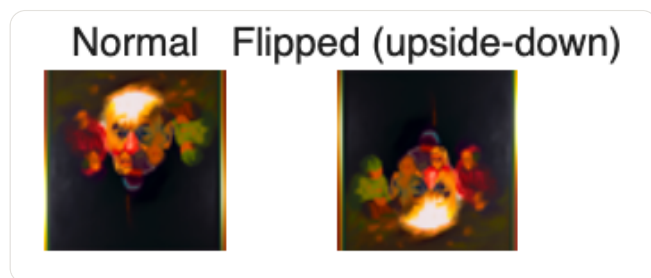
output/
partA/1.7.3_my_img_2.png

PART A • SECTION 1.8

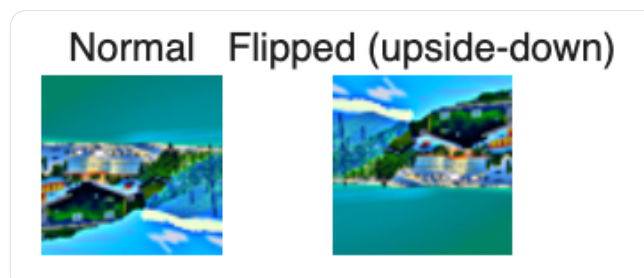
Visual Illusions

Visual anagrams are constructed so that one interpretation appears in the original orientation and a second interpretation appears after a transformation. Below are two illusion results included for this part.

Two Illusions of Choice



Visual illusion 1. `output/partA/1.8_img_1.png`



Visual illusion 2. `output/partA/1.8_img_2.png`

PART A • SECTION 1.9

Hybrid Images and Roar-ee Translation

Hybrid images combine coarse low-frequency structure from one image with high-frequency details from another. This section also includes the Roar-ee image-to-image result that was generated for Columbia's mascot prompt.

Hybrid Images

Hybrid image 1. `output/partA/1.9_img_1.png`Hybrid image 2. `output/partA/1.9_img_2.png`

2.0 Roar-ee Image-to-Image Translation

roaree_hipster_original roaree_hipster_i_start=1 roaree_hipster_i_start=3 roaree_hipster_i_start=5 roaree_hipster_i_start=7 roaree_hipster_i_start=10 roaree_hipster_i_start=20



Columbia Roar-ee themed image-to-image translation result.

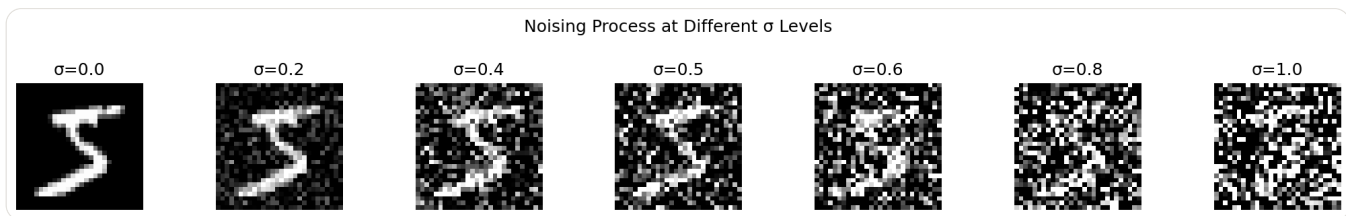
`output/partA/2_roaree_skull.png`

PART B • SECTION 1.2

Visualizing the Noising Process

Clean MNIST images are corrupted via $z = x + \sigma \epsilon$ where $\epsilon \sim N(0, I)$. We visualize $\sigma \in \{0.0, 0.2, 0.4, 0.5, 0.6, 0.8, 1.0\}$ on a single digit.

Noising at Different σ Levels



A single MNIST digit corrupted at $\sigma = 0.0 \rightarrow 1.0$ (left to right).

`output/partB/1.2_noising_process.png`

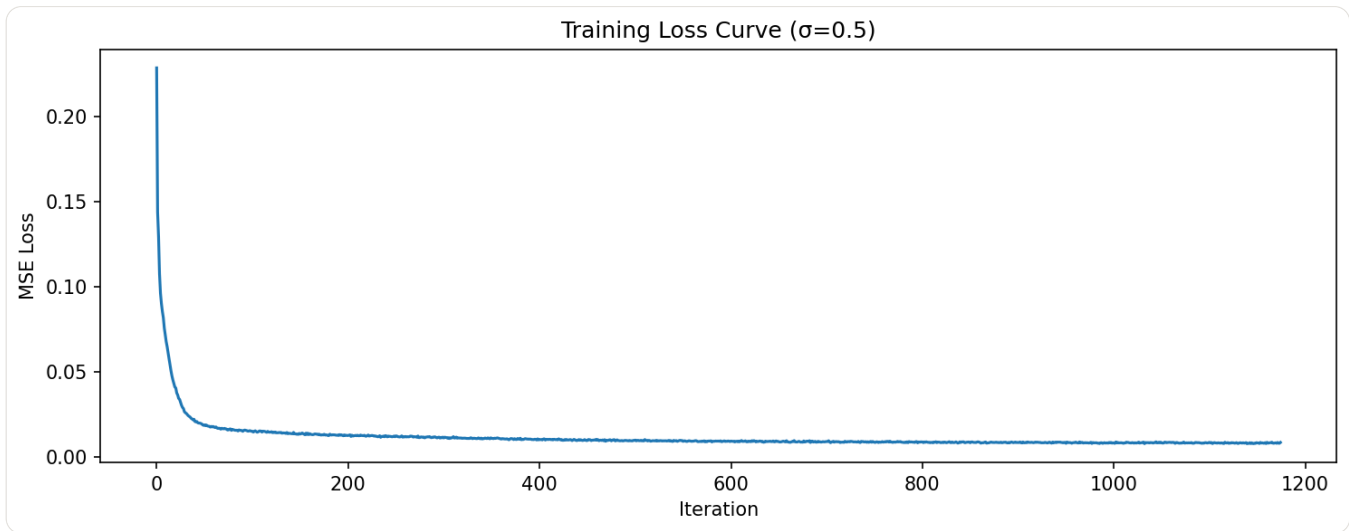
PART B • SECTION 1.2.1

Training the Denoiser ($\sigma = 0.5$)

An unconditional UNet is trained to map noisy images ($\sigma = 0.5$) back to their clean counterparts using an L2 loss. Noise is sampled fresh each iteration for better generalisation.

Architecture	UnconditionalUNet, $D = 128$
Optimizer	Adam, $\text{lr} = 1\text{e-}4$
Batch size	256
Epochs	5
Noise level σ	0.5

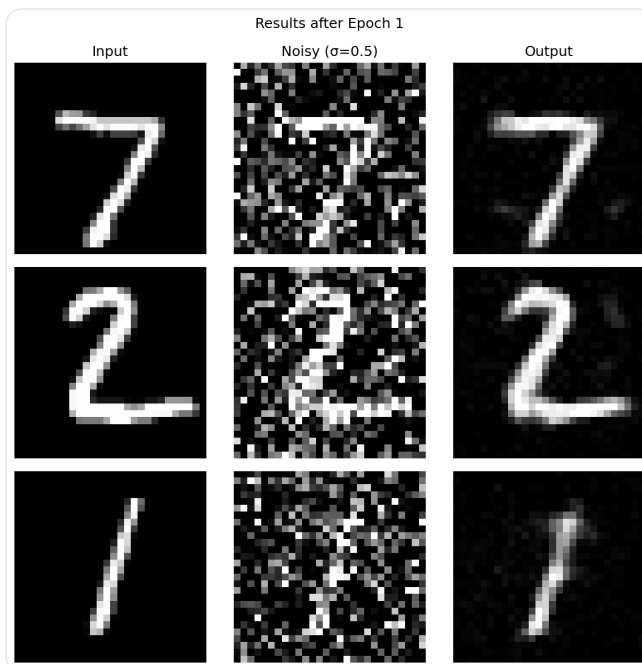
Training Loss Curve



MSE loss across all 5 epochs of $\sigma = 0.5$ denoising training.

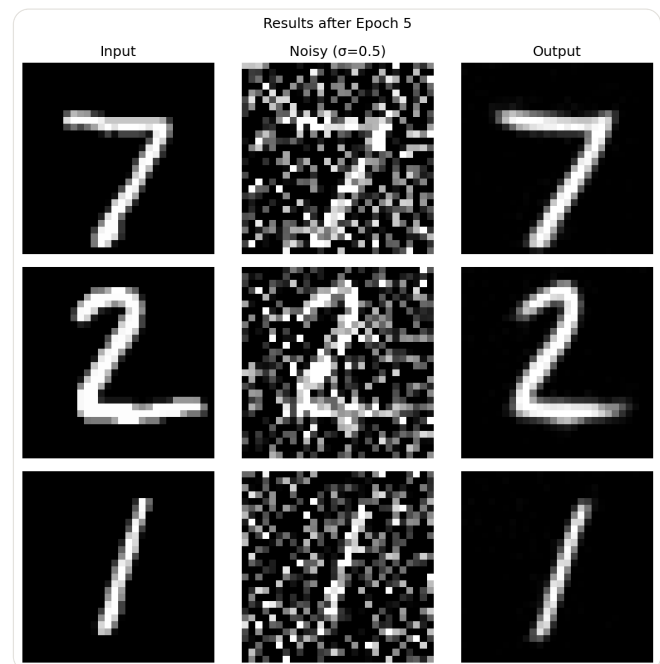
`output/partB/1.2.1_training_loss_curve.png`

Denoising Results — Epoch 1 & Epoch 5



Epoch 1 — coarse structure recovered but outputs are slightly blurry.

`output/partB/1.2.1_denoising_epoch_1.png`



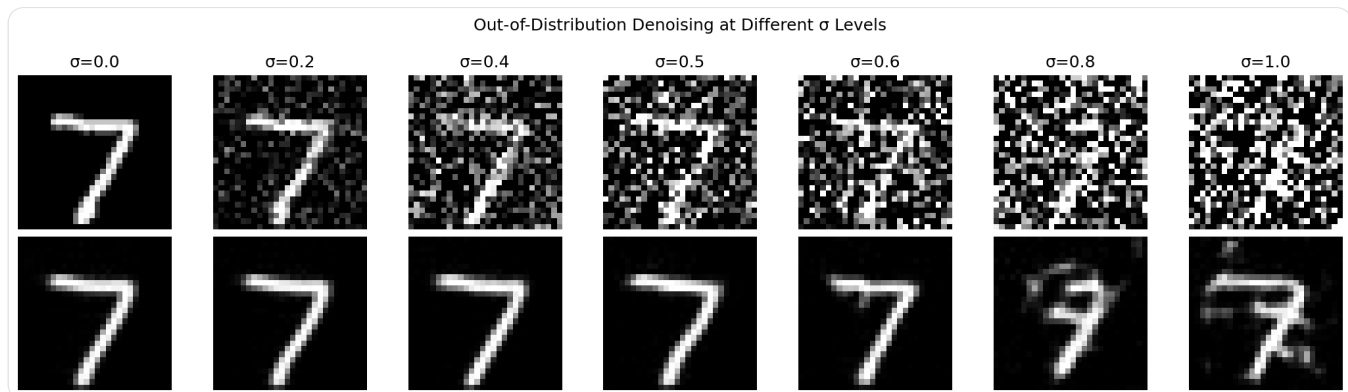
Epoch 5 — sharper and more faithful to the original digits.

`output/partB/1.2.1_denoising_epoch_5.png`

Out-of-Distribution Noise Testing

The denoiser trained at $\sigma = 0.5$ is evaluated on $\sigma \in \{0.0, 0.2, 0.4, 0.5, 0.6, 0.8, 1.0\}$ to understand how performance degrades outside the training distribution.

OOD Denoising Results



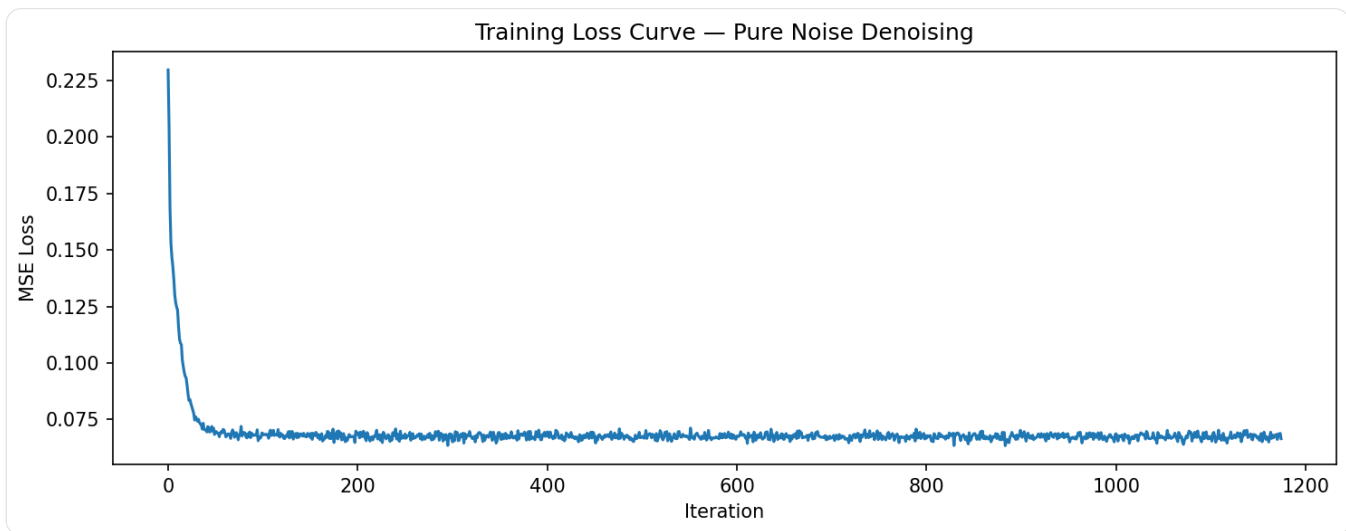
Top row: noisy inputs. Bottom row: denoiser outputs. Performance degrades at extremes — over-smoothing at low σ , failing at high σ . [output/partB/1.2.2_ood_testing.png](#)

PART B • SECTION 1.2.3

Denoising Pure Noise

The UNet is retrained with pure Gaussian noise $\epsilon \sim N(0, I)$ as input and a clean MNIST digit as target. Since the input carries no class information, the model learns to output the *mean* of the training distribution.

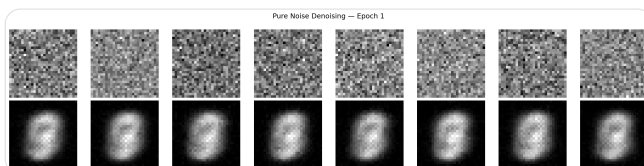
Training Loss Curve — Pure Noise



Loss drops sharply then plateaus at ~ 0.07 — the irreducible variance of MNIST around its mean.

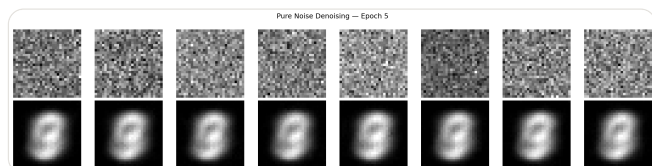
`output/partB/1.2.3_training_loss_pure_noise.png`

Sample Results — Epoch 1 & Epoch 5



Epoch 1 — model begins converging toward a mean-digit shape.

`output/partB/1.2.3_pure_noise_epoch_1.png`



Epoch 5 — every input maps to the same blurry digit-like blob.

`output/partB/1.2.3_pure_noise_epoch_5.png`

Observation: All outputs collapse to a single blurry averaged digit regardless of the input noise. With MSE loss the optimal unconditional predictor is the mean of all training images — the centroid of the digit distribution. This motivates flow matching and class conditioning in Part 2.

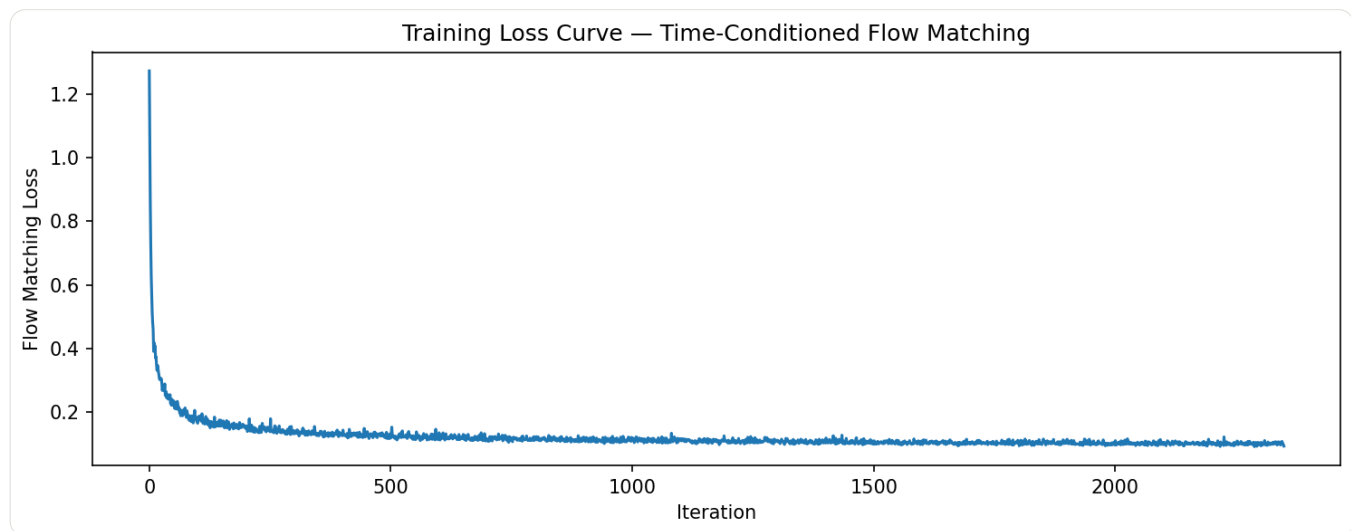
PART B • SECTION 2.2

Training the Time-Conditioned UNet

Flow matching trains $u_\theta(x_t, t)$ to predict the velocity field $x_t - x_0$ along linear interpolations between noise and clean data. Euler integration from $t = 0 \rightarrow 1$ recovers a realistic sample at inference.

Architecture	TimeConditionalUNet, $D = 128$
Conditioning	Scalar t via 2 FCBlocks
Optimizer	Adam, $\text{lr} = 1\text{e-}4$
Epochs	10
Timesteps (inference)	50

Training Loss Curve



Flow matching loss (MSE on predicted vs. true velocity) across 10 epochs.

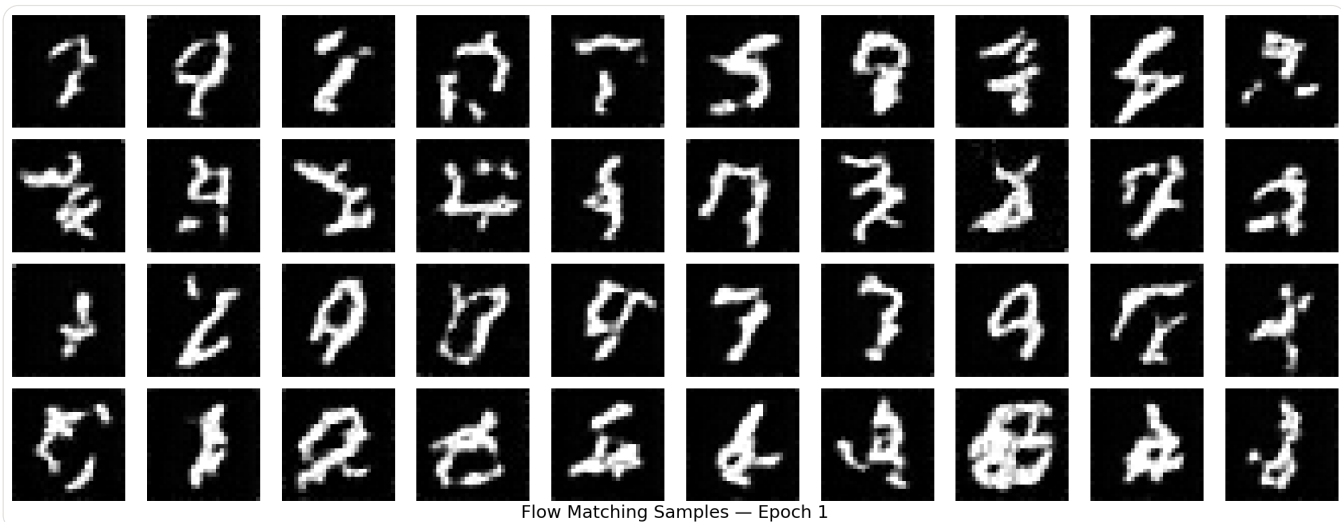
`output/partB/2.2_fm_training_loss.png`

PART B • SECTION 2.3

Sampling from the Time-Conditioned UNet

Starting from pure Gaussian noise, 50 Euler steps integrate the learned velocity field. Results shown after 1 and 10 epochs of training.

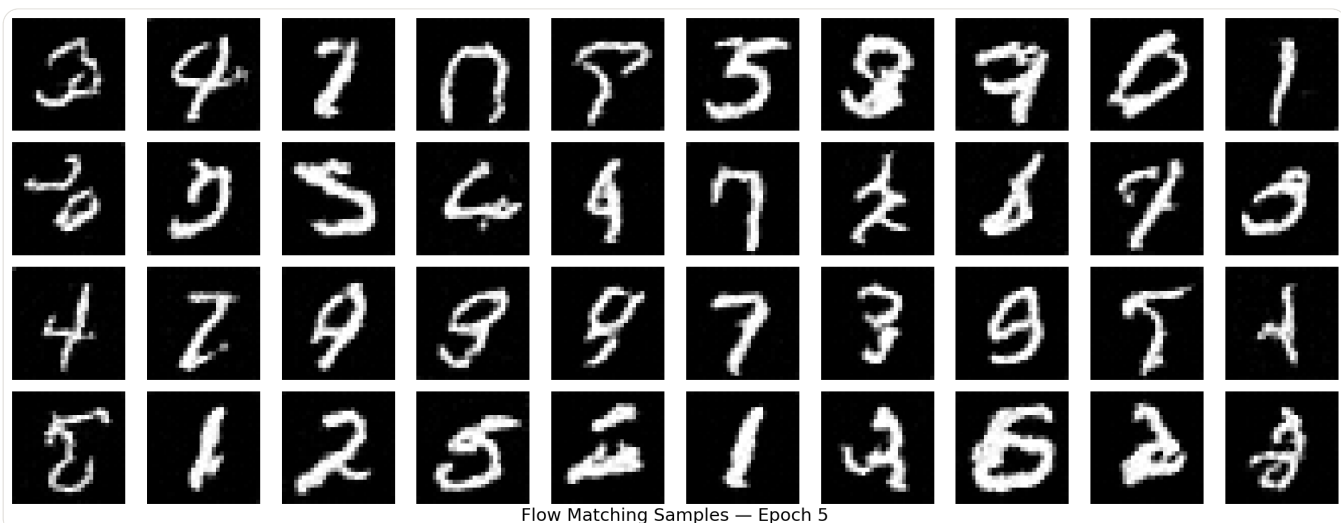
Generated Samples — Epoch 1



Epoch 1 — outputs are noisy and loosely digit-shaped.

`output/partB/2.3_fm_samples_epoch_1.png`

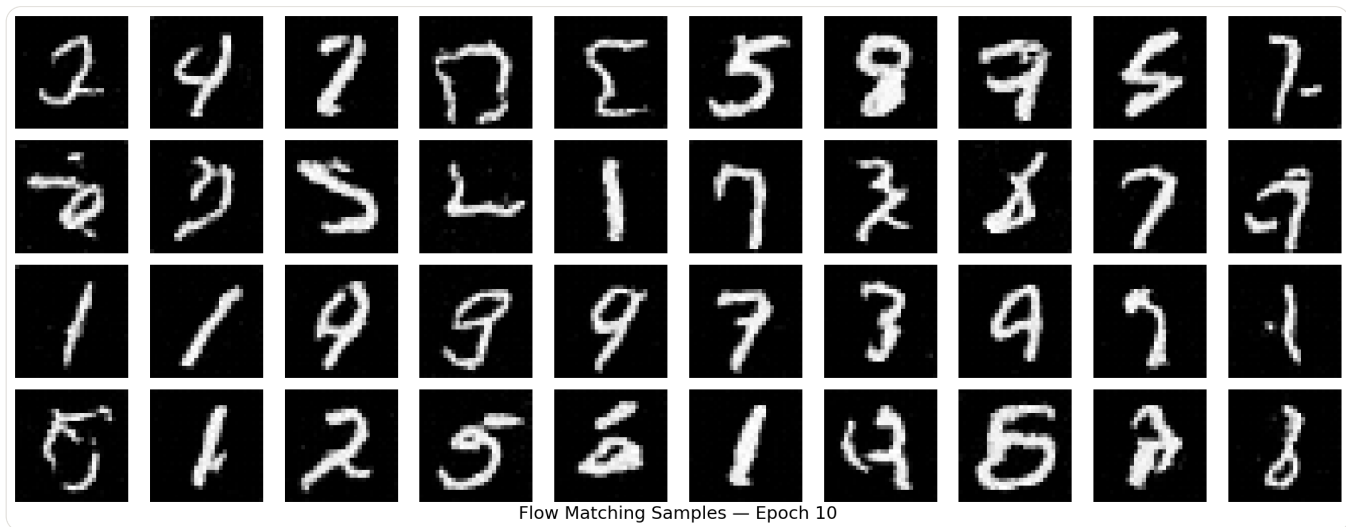
Generated Samples — Epoch 5



Epoch 5 — recognisable digit shapes emerge with improved sharpness and diversity.

`output/partB/2.3_fm_samples_epoch_5.png`

Generated Samples — Epoch 10



Epoch 10 — samples are clearly digit-like, randomly distributed across 0–9.

`output/partB/2.3_fm_samples_epoch_10.png`

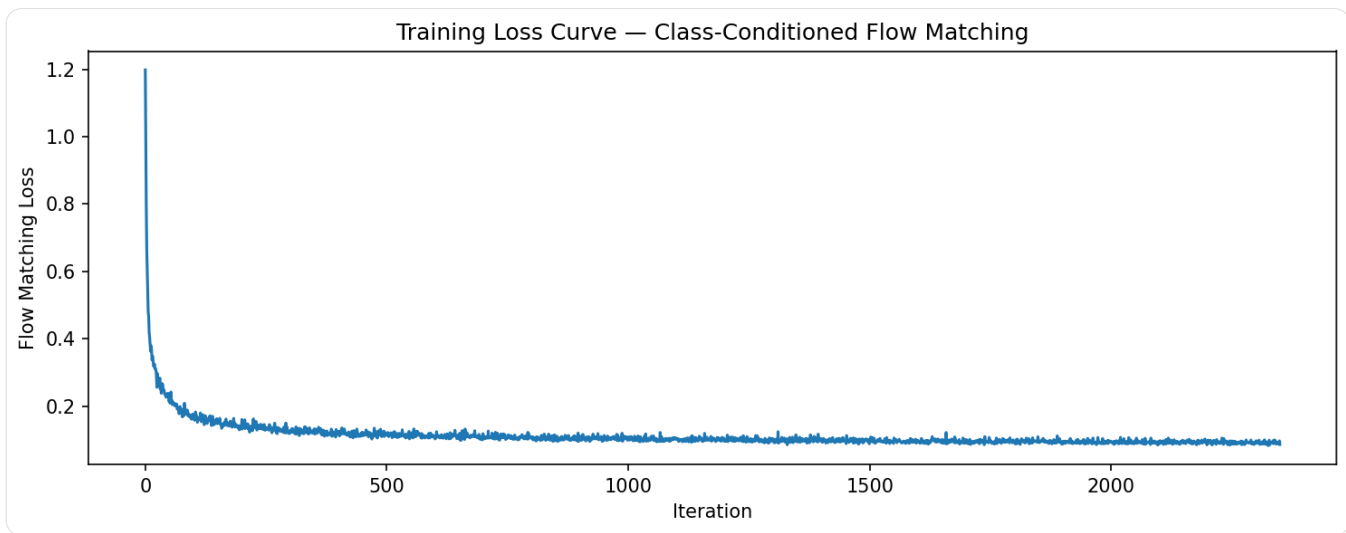
PART B • SECTION 2.5

Training the Class-Conditioned UNet

The UNet is extended with class conditioning via one-hot digit labels (0–9), injected through two additional FCBlocks. CFG dropout ($p_{\text{uncond}} = 0.1$) enables unconditional generation.

Architecture	ClassConditionalUNet, D = 128
Conditioning	t + c (one-hot, 10 classes) via 4 FCBlocks
CFG dropout p_{uncond}	0.1
Guidance scale γ	5.0 (inference)
Optimizer	Adam, $\text{lr} = 1\text{e-}4$
Epochs	10

Training Loss Curve



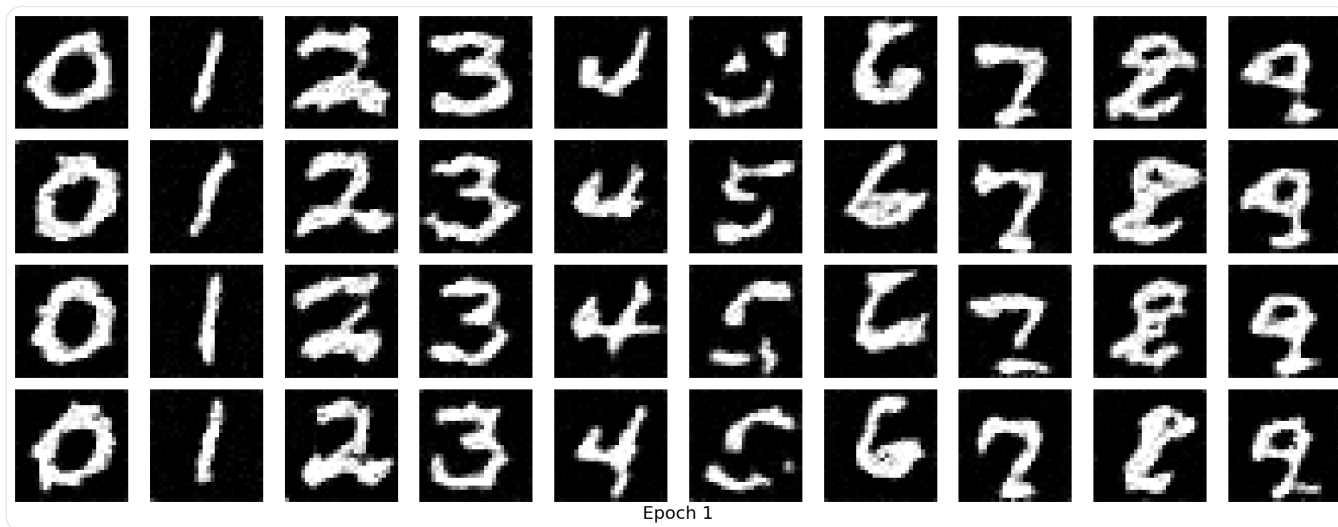
Flow matching loss for the class-conditioned model across 10 epochs. Convergence is faster than the unconditional model. [output/partB/2.5_class_fm_training_loss.png](#)

PART B • SECTION 2.6

Sampling from the Class-Conditioned UNet

Using classifier-free guidance ($\gamma = 5.0$), four instances of each digit 0–9 are generated. Each column = one digit class. Results shown after 1, 5, and 10 epochs.

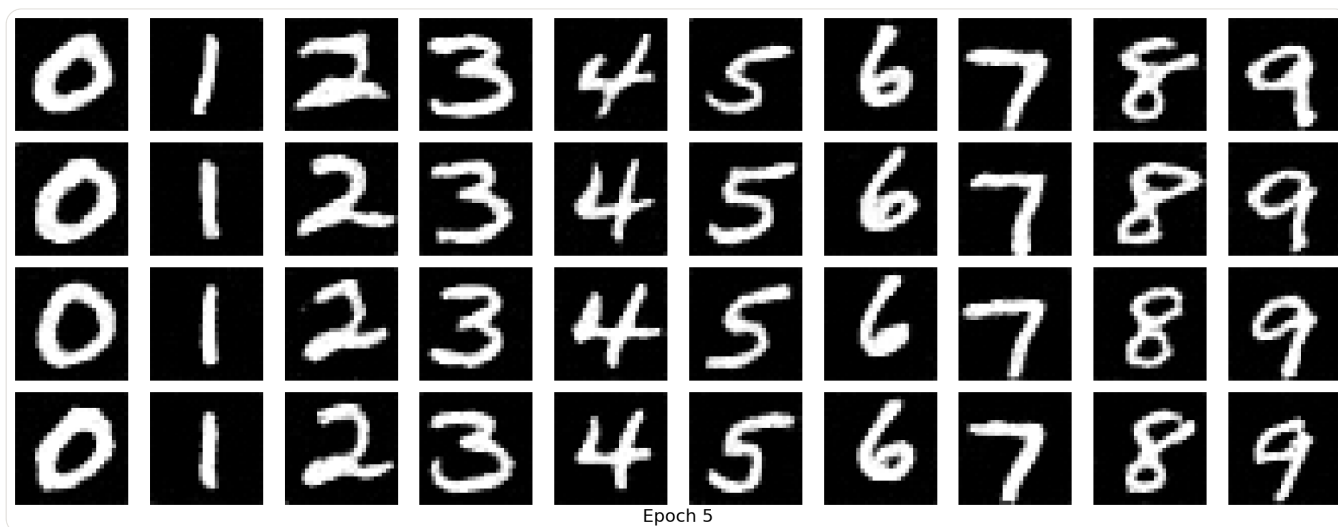
Generated Samples — Epoch 1



After epoch 1 digit classes are already partially separable with CFG.

`output/partB/2.6_class_fm_samples_epoch_1.png`

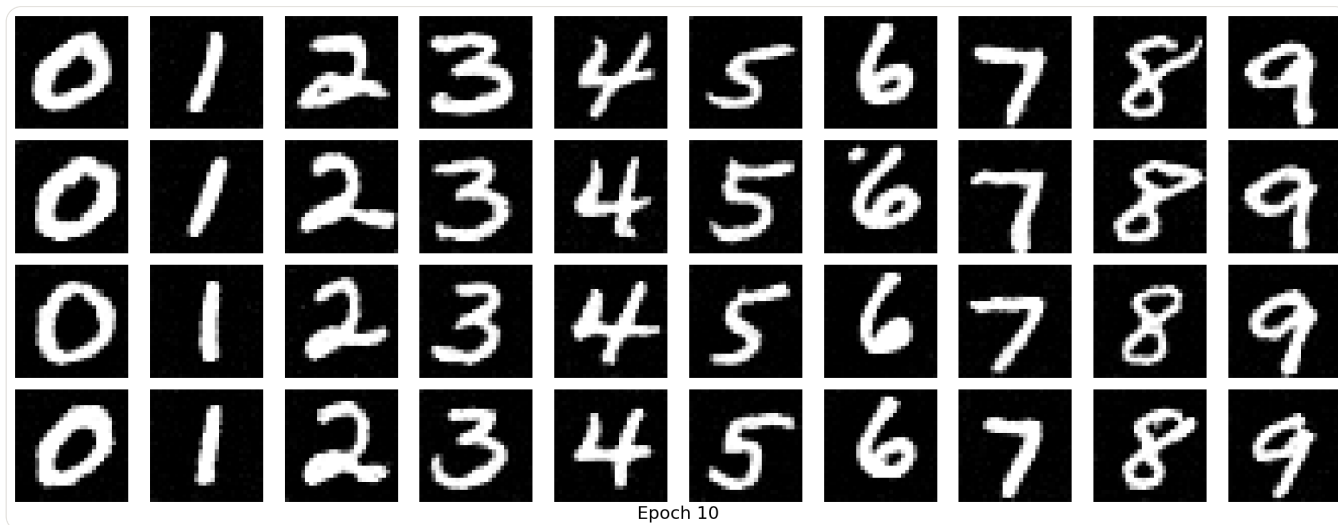
Generated Samples — Epoch 5



By epoch 5 digit identity is clear for most classes.

`output/partB/2.6_class_fm_samples_epoch_5.png`

Generated Samples — Epoch 10



After 10 epochs samples are sharp, recognisable, and well-separated by class.

`output/partB/2.6_class_fm_samples_epoch_10.png`

Learning Rate Scheduler: We did not use a learning rate scheduler in any part of this project. A fixed Adam optimizer with $lr = 1e-4$ was sufficient to achieve stable convergence throughout all training runs. The fixed learning rate served as an implicit form of regularisation and no scheduler was needed to compensate.